AUFBAU STUFE 1

Keine Vererbung, nur „hat ein“.

Klasse	hat	Methoden
Lager	viele Produkte	`anzeigen`, `finden`, `suchen`
Warenkorb	Kunde, Positionen	`hinzufuegen`, `entfernen`, `gesamtsumme`, `ist_leer`, `anzeigen`
Position	Produkt, Menge	`summe`

Startpaket: `produkt.py` und `daten.py` nur benutzen, `main.py` ausbauen (Menü mit `while True` und `match`).

MUSTER FINDEN

```
def finden(nummern, gesucht):
    for n in nummern:
        if n == gesucht:
            return n    # sofort raus
    return None         # nach der Schleife!

liste = [101, 102, 203, 204]
print(finden(liste, 203))  # 203
print(finden(liste, 999))  # None
```

Runde	n	== 203?	was passiert
1	101	nein	weiter
2	102	nein	weiter
3	203	ja	`return`, Ende

Merksatz: „Nicht gefunden“ darfst du erst sagen, wenn du alle angeschaut hast.

MUSTER SAMMELN

```
def suchen(namen, text):
    treffer = []
    for name in namen:
        if text.lower() in name.lower():
            treffer.append(name)
    return treffer

namen = ["Kaffee 1 kg", "Grüner Tee",
         "Kaffeemühle"]
print(suchen(namen, "kaffee"))
# ['Kaffee 1 kg', 'Kaffeemühle']
print(suchen(namen, "Zucker"))  # []
```

Leere Liste vor, `append` in, `return` nach der Schleife. Kein Treffer = `[]`, nicht `None`.

HINZUFUEGEN() SCHRITT FÜR SCHRITT

1. `schon_im_korb` = Menge, die schon im Korb liegt.
2. `schon_im_korb + menge > bestand`? Dann `return False`, nichts ändern.
3. Noch keine Position: neue `Position` anhängen. Sonst `position.menge += menge`. Dann `return True`.

Bestand 3:

Aufruf	im Korb	Prüfung	Ergebnis
menge 5	0	0 + 5 > 3	False
menge 3	0	0 + 3 > 3 nein	True
menge 1	3	3 + 1 > 3	False

`hinzufuegen()` liefert nur `True` / `False`, die Meldung schreibt `main.py`.

MUSTER AUFSUMMIEREN

```
summe = 0
for p in [74.61, 46.38]:
    summe += p
print(round(summe, 2))  # 120.99
```

Im Projekt: `summe = Decimal(0)`, dann `summe += position.summe()`.

MENÜ UND BESTELLEN

- Lager, Kunde, Warenkorb **vor** der `while`-Schleife anlegen.
- Bestellen: leerer Korb → Meldung, `return`. Sonst für jede Position `verkaufen(menge)`, Summe ausgeben, Korb leeren.
- Frühes `return` beendet nur den Menüpunkt.

```
eingabe = "abc"
print(eingabe.isdigit())  # False
print("999".isdigit())    # True
```

STUFE 2: POLYMORPHIE

`Produkt` wird abstrakt mit `mwst_satz()`. `Lebensmittel`: 7 %, `Zubehoer`: 19 %. `bruttopreis` steht nur einmal in `Produkt` und ruft `self.mwst_satz()`.

```
from decimal import Decimal
p = Decimal("12.99") * Decimal("1.07")
print(p.quantize(Decimal("0.01")))  # 13.90
```

NAMEN ZEIGEN AUF OBJEKTE

```
korb = ["2 x Kaffee"]
bestellung = korb      # keine Kopie
korb.clear()
print(bestellung)      # []
korb = ["1 x Tee"]
best2 = korb
korb = []              # neue Liste
print(best2)           # ['1 x Tee']
```

Merksatz: `clear()` leert das Objekt für alle Namen, `= []` bindet nur einen Namen neu.

TYPISCHE FEHLER

- `else: return None` direkt unter dem `if`: Suche endet nach dem ersten Produkt.
- `return None` in der Schleife statt danach.
- Nur `menge > bestand` geprüft, Menge im Korb vergessen.
- Immer `append`: dasselbe Produkt als zwei Positionen.
- `korb = Warenkorb(kunde)` in der Schleife: Korb jedes Mal leer.
- `korb.positionen.clear()` leert auch die Bestellung, richtig `korb.positionen = []`.
- Kein Treffer als `None` statt `[]` zurückgegeben.